

AI for Economic Research: Dynamic Models, Language, and Agents

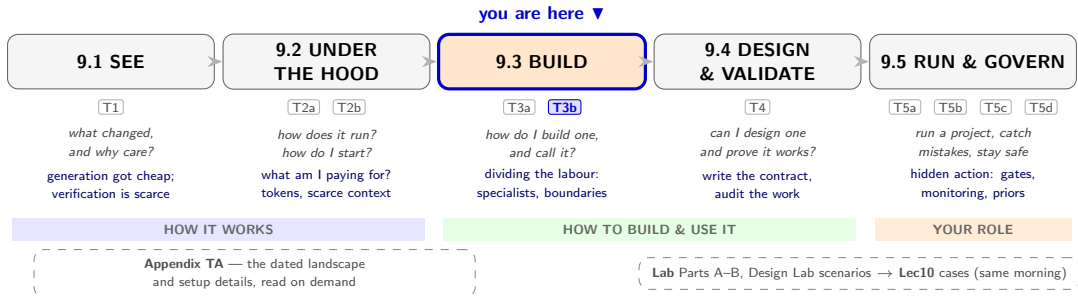
Lecture 9.3b: Invoking Agents — One, Three, Ten Thousand

Zhigang Feng

2026-09-12 16:01:42

Lecture 9 in One Map: Invoking Agents

T3a left three job descriptions in a folder. This deck answers what you type to make one run — and where that leads.



The claim this deck defends. The number of agents you run is not a capability decision. It is a *verification* decision — you may only buy scale when a wrong answer is cheap to detect.

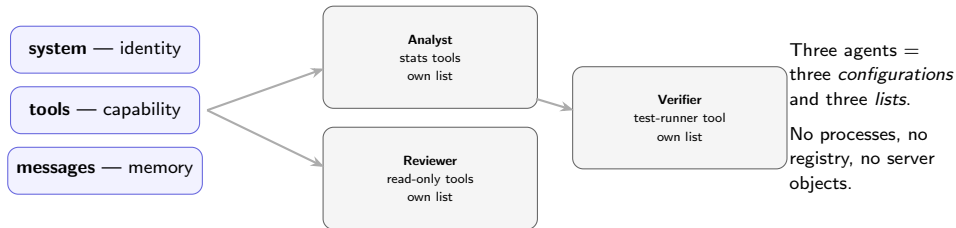
Reading convention (Appendix TA): [solid] = primary or citable; [hype] = vendor self-report.

Many Agents

Creating a set of them — and what actually limits the number

Three Dials Make an Agent

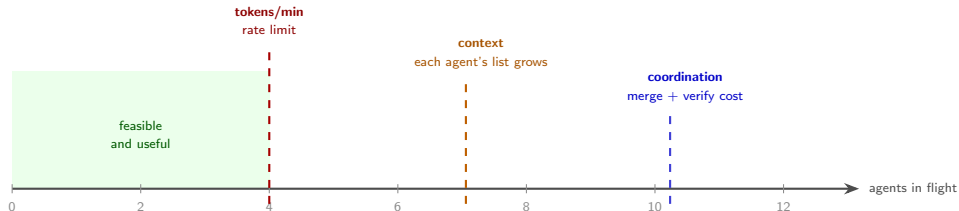
To create a set of agents you do not create anything. You vary three dials and keep three separate lists.



The economist's reading. You are not hiring staff; you are writing three job descriptions and keeping three notebooks. The “firm” exists only in your code. What is scarce is not agents — it is tokens, context, and your attention.

How Many at Once? Three Constraints, Not Infinity

“You can instantiate an unlimited number of agents” is true in the same sense that you can write an unlimited number of cheques.

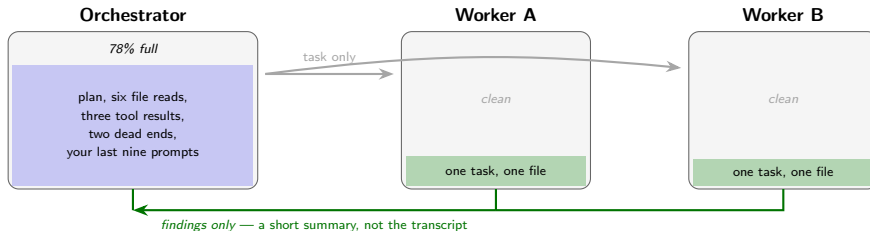


- **Rate limits** are the hard ceiling: quota is tokens-per-minute, not agents. Ten agents each re-sending a long list exhaust it faster than two.
- **Context** binds per agent: every worker carries its own growing list (previous section).
- **Coordination** is the one economists should expect: *you* must read, reconcile, and verify n outputs. That cost rises with n while the value of the marginal agent falls.

The binding constraint is usually the third one. T1 priced this out: multi-agent bought a large accuracy gain at roughly $15\times$ the token cost. Reach for more agents when the task is genuinely wide, not to feel parallel.

Isolated Context Is the Point

The reason to spawn a sub-agent is not extra brains — the model is the same. It is a second, clean list.

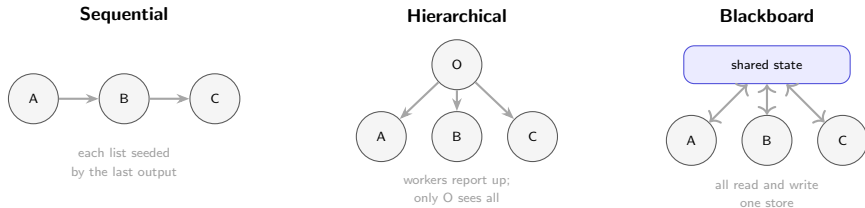


- The worker never sees your dead ends, so cannot be misled by them
- The orchestrator's list grows by a summary, not by all the worker read
- That asymmetry is the economic case for sub-agents

Do not over-read this. A clean window is not an independent mind. T5c shows workers inherit your framing — fresh context buys *focus*, not *independence*.

Three Wirings, Drawn as Data Flow

Strip the vocabulary away and multi-agent designs differ in exactly one respect: who writes into whose list.



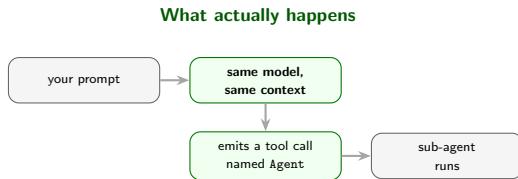
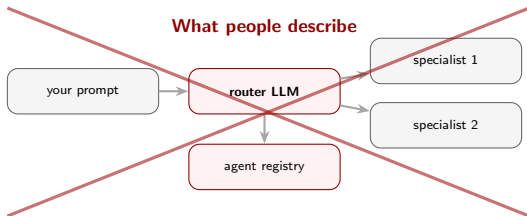
- **Sequential** is a relay: cheap, easy to debug, no parallel gain, and an early error propagates
- **Hierarchical** is T1's orchestrator-worker and what T5b's RA team is — the pattern that dominates in practice
- **Blackboard** decouples agents from each other but needs write discipline; in a research project the "blackboard" is usually just *files on disk* — exactly the file-driven workflow of T5b

Who Decides to Dispatch?

The router myth, and what the transcript actually shows

The Router Myth

The usual explanation: a fast “router” model classifies your prompt, looks up specialists in an agent registry, and wakes them. That is not what happens.



- **No second model, no classification step.** Dispatch is one more `tool_use` block from T2a's loop — the tool just happens to start another loop
- The “registry” is a folder of Markdown files *you wrote*; the model picks from it by reading their description fields, exactly as it picks any tool
- T2a said this for one agent: *“the same transformer that writes code decides to call tools.”* This is that fact one level up

Guard: TA's “think-budget routers” route one query between fast and deep modes of one model, not between agents.

Dispatch Is a Tool Call — Evidence You Can See

You do not have to take this on faith. Four checks, all runnable in the lab that follows.

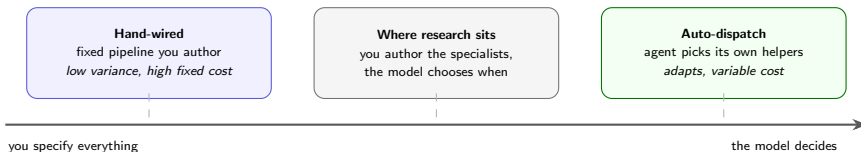
	Do this	What it proves
1	Run <code>/agents</code> in a session	The list is <i>your</i> <code>.claude/agents/</code> folder. Nothing is registered with a service.
2	Ask something a specialist fits; watch the transcript	The dispatch appears as a named tool call in the same stream as <code>Read</code> and <code>Bash</code> .
3	Compare context before and after	It grows by a short <i>summary</i> , not by the worker's transcript.
4	Narrow one agent's description, re-ask	Dispatch stops. Selection is driven by prose the model reads.

Check 4 is the decisive one. If a separate router with a registry were making the decision, editing one English sentence in a Markdown file could not switch the behaviour off. It can — because that sentence *is* the mechanism.

Real trace, check 2 (Claude Code 2.1.261): `Read` → *Agent* `code-reviewer` → *Agent* `domain-reviewer` — the dispatch sits in the same list as `Read`. **[solid]**

Auto-Dispatch vs. Hand-Wired: Spontaneous Order or Central Planning

Once you know dispatch is a model decision, the design choice becomes familiar: leave allocation to the agent, or plan it yourself.



Central planning (hand-wired)

- Reproducible: same path every run
- No planner tokens spent
- Brittle off the anticipated path
- High authoring cost; it decays

Spontaneous order (auto-dispatch)

- Handles tasks you did not foresee
- Zero setup at the point of use
- **Principal-agent problem:** it may delegate the wrong thing, and you pay
- Variable, harder-to-forecast cost

The practical middle. Author the specialists once, let the model choose *when* to call them, and keep a validation gate so a bad delegation is caught rather than trusted — T5b's quality gates, doing an economist's job.

Three Rungs

The only three ways to invoke an agent — and who chooses in each

The Question T3a Left Open

Three job descriptions sit in a folder. Nothing yet runs them. There are exactly three ways to start one, and they differ in one thing: who chooses.



Only the third rung scales, because only it removes you from the loop. Rungs 1 and 2 cost a turn of your attention each — the third ceiling above, the one that binds. Rung 3 trades that for a prior problem: what checks the output.

Answered next: the exhibit the router section argued for but never showed.

Rung 1 — You Describe It, the Model Chooses

The router section proved dispatch is an ordinary tool call, by argument and by experiment. It never showed you one. Here it is.

```
you > Check the derivation in src/aiyagari.py
      against docs/model.md.

# what the model emits -- one block in the same
# list as Read and Bash, nothing special about it
{ "type": "tool_use",
  "name": "Agent",
  "input": {
    "subagent_type": "math-agent",
    "description": "check the derivation",
    "prompt": "Review src/aiyagari.py against
               docs/model.md. The calibration is
               annual. Report only findings you
               can tie to a line number." } }

# ... the sub-agent's whole session happens here ...

{ "type": "tool_result",
  "content": "line 34: utility() has no domain
             guard. With SIGMA=2, u(c)=-1/c is
             positive for c<0 ..." }
```

- **You typed no agent name.** The model read the description fields in your folder and chose — the same way it chooses Read
- **prompt is the brief,** T3a's step 4, written by the model this time rather than by you
- **One tool_result comes back.** Everything the sub-agent read is gone — T3a's step 5

The same thing in a real transcript, from the lab: Bash → Agent code-reviewer → Read — Agent sits in the same list as the others. That is the whole claim. [solid]

Answered next: what changes when you name it yourself.

Rung 2 — You Name It

Naming the agent moves the choice from the model to you. Three surfaces do it, and this course already uses two of them without saying so.

```
# (a) in prose -- the idiom the OLG demo already uses
you > Send the pseudo-code to the domain-reviewer
      sub-agent; require it to trace one cohort's
      residual end to end before I write code.

# (b) inspect what is actually in scope
/agents

# (c) pin a whole session to one specialist
claude --agent math-agent

# (d) define one inline, with no file at all
claude --agents '{"claim-checker":
  {"description":"Find the source of every number.",
   "prompt":"You verify figures against the repo."}}'
```

/agents is the one command T2b's twelve do not list; T2b owns the rest of the surface.

Answered next: the rung that removes you, and reaches four digits.

Naming it skips the only test you had. Rung 1 makes the model choose by reading your description. That choice *is* the test that the description is right — T3a's step 7 is built on it.

Name the agent every time and a badly worded description never fails, so you never find out. Use rung 2 when you know which specialist you want; use rung 1 when you are still debugging the English.

Rung 3 — You Script It

Rungs 1 and 2 spend a turn of your attention per dispatch. Rung 3 spends a process instead — and it is the only rung that reaches four digits.

```
# one process per unit of work, fenced read-only
for f in source/Lec09_Agentic_AI/Lec09_T*.tex; do
  claude -p "$(cat briefs/claim_check.md) $f" \
    --output-format json \
    --allowedTools Read Grep Glob \
    --permission-mode dontAsk \
    > "out/$(basename "$f" .tex).json" &
done
wait
# or hand them to the background control plane
claude --bg "audit T3b for unsourced numbers"
claude agents          # list them
claude logs <id>       # read one
claude stop <id>       # kill one
```

Rung	Costs you	Binds at
1 implicit	a turn, each time	your attention
2 explicit	a turn, each time	your attention
3 scripted	one brief, once	tokens per minute

The brief becomes a file. At rung 3 you cannot retype it, so it must live on disk — T3a's step 8, arriving by necessity rather than discipline.

Choosing a Rung Is One Question

Not “how many agents can I run” — The first section of this deck already answered that. The question is what checks the output, and how cheaply.

What can check the output	Rung	Worked example
only you, by reading	1–2, N small	the three background specialists, judged by a 3×3 detection matrix (T3a)
a cheap partial check — tests, a rubric, a schema — plus a tier for the hard cases	3, N in the thousands	T6b: cached rubric, Sonnet adjudication, 11,171 labels
a sound mechanical check	3, N until the quota binds	Lean, a compiler, a one-command replication script

N is a verification choice, not a capability choice.

The ceilings tell you where the limit is. The verifier tells you whether you want to go near it.

Four-Digit Runs

What a four-digit agent run consists of — and what made it bankable

What About Ten Thousand Agents?

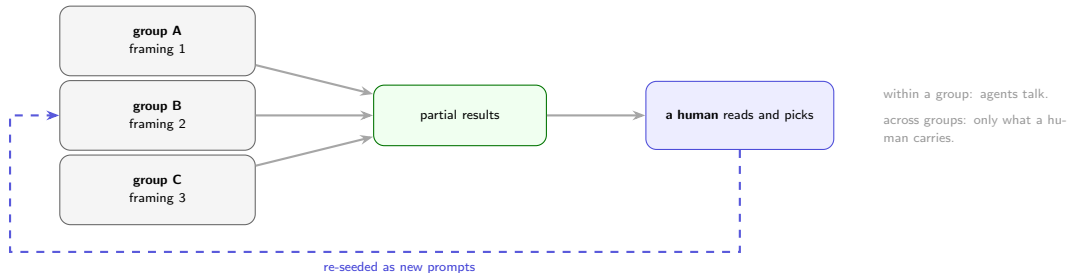
On 28 August 2026 OpenAI pointed roughly ten thousand agents at Navier–Stokes and had a Lean-checked proof 88 hours later. The number is the least interesting part.

Warm-up (Euler)	~100 agents, ~50 hours
The run (Navier–Stokes)	~10,000 agents from 28 Aug 2026, done within 88 hours
Formalisation	a further 17 hours to check the proof in Lean
Traffic	~5 million messages <i>between agents</i> ; ~300 billion output tokens
Compute	“several million dollars” (Bubeck, OpenAI)
What was shown	finite-time singularity from smooth data — the <i>negative</i> resolution

Read the ledger, not the headline. Every figure is the vendor’s own account of an unreleased model, and the result is disputed two frames on. **[hype]** What is worth your attention is the *architecture* — reported consistently, and the part that transfers.

Not Ten Thousand Geniuses: Groups, Framings, and a Human

Not ten thousand independent attempts. Agents sat in groups that talked internally, each group got a different framing of the same problem, and a human moved fragments between groups.



What varied was dial one. Different framings of one problem *are* different system prompts — T3a's step 2, run ten thousand times instead of three — wired as the blackboard above with a person as the board. Sober reading: a thousand-agent run with better synthesis might land in the same place.

The Verifier Is the Whole Story

You may only spend money on scale when a wrong answer is cheap to detect. Lean made a wrong proof cost nothing. That, and not the ten thousand, is the part that transfers.

Reporting best-of- n as your score	the sin T4 names: τ -bench pass-hat- k falls from $\sim 90\%$ at $k=1$ to $\sim 57\%$ at $k=8$, so a best-of- n headline flatters the model and hides the reliability curve
Running n attempts behind a checker that can reject	legitimate — because the <i>checker</i> decides, not the score. The 9,999 failures cost tokens and nothing else

Two things Lean does not buy. A human must still certify that the statement checked in Lean is the theorem anyone meant to prove — the formal check is sound about the object it was given, not about your intent. And the result is contested: Buckmaster alleges OpenAI muscled in on work he and Alpöge were finishing, called one AI-written paper “AI slop” *despite* its formal verification, and OpenAI denies the charge. **[solid]** on the dispute.

Answered next: you already own a run like this — and it worked for exactly this reason.

Your Own Four-Digit Run, and Why It Worked

Lec10's T6b labelled 11,171 of 13,225 tasks in a single five-hour window. Nobody called it a swarm, but it is the same machine — and it had a verifier.

	Navier–Stokes	T6b, your labelling run
The checker	Lean: sound, mechanical	a rubric, cached per task
The hard cases	re-framed and re-seeded by a human	a Sonnet adjudication tier for the 3,456 flagged
Not repeating work	—	one cache file per task; re-runs skip
Human time	re-seeding between groups	~20 min to spec the rubric; ~12 labels read by eye

Rubric + adjudication tier + cache is a verifier — weaker than Lean, cheap enough to run 13,000 times. **And note what neither buys:** you can re-run a proof attempt or a labelling task ten thousand times, but you cannot re-run 2008. A central bank gets one history, so fan-out never manufactures a counterfactual (Lec05).

T6b owns these numbers and the bulking/caching design; this frame owns only the mapping. [solid]

The Ledger

What four sub-agents cost — the bill, and the design work

The Ledger: What Four Sub-Agents Cost

The thirteen frames of T3a's eight-step section were not written by one session. Four agents did the reading. Here is the bill, from this project's own logs.

Type	Brief	Tokens	Calls	Min
Explore	map the target deck and its house style	57,474	15	3.0
Explore	build system, CJK support, TikZ idioms	121,309	43	5.0
Explore	course context, capstone, PDF tooling	145,432	34	5.6
Plan	draft the frame-by-frame outline	124,361	30	10.3
total		448,576	122	23.8

448,576 tokens were read; four summaries came back — roughly twenty- to thirty-fold compression, and the orchestrating session never held the 122 file reads that produced them. That is the isolated-context argument above with a price on it. The three explorers also ran *concurrently*: **5.6 minutes of wall clock against 13.5 sequential** (T5b owns the general speed claim).

The Design Work Was the Four Briefs

Dispatching cost one line each. What decided whether the deck was any good was what each brief asked for — T3a's step 4, done four times.

Asked for	Came back	Changed
sections, frames, verbatim style patterns	no deck shows an agent file	the section's premise, and the style contract
engine, CJK support, best existing figures	a Chinese guard exists in Lec01; zero <code>includegraphics</code> ; two TikZ idioms	staying on <code>pdflatex</code> ; both card figures
capstone, prerequisites, PDF tooling	prerequisites require neither Python nor ML; a converter pipeline exists on disk	claims 1 and 4; the pipeline frame

None of the three asked a question whose answer was already known. Each asked for an *inventory* — what exists, where, quoted — which is what a read-only agent with a clean window is good at. Ask an explorer for a *judgement* and you get your own prior back with file paths attached (T5c).

Synthesis

Two artifacts: a folder of files, or a loop over processes

The Corrections, in One Table

Everything here is checkable on your own machine — the vocabulary is marketing, the mechanism is not.

Widely repeated	What the machine does
"The session remembers me."	The API is stateless. A session is a list you re-send in full every turn — your cost curve and your context limit.
"A router dispatches specialists from a registry."	The same model, in the same context, emits a tool call. The registry is your folder of Markdown files.
"A skill is hard-coded software."	A skill is prose the model reads, disclosed progressively. Its description is what selection runs on.
"You can run unlimited agents."	Tokens-per-minute, context growth, and <i>your</i> coordination cost bind — usually the third.
"CLAUDE.md is just documentation."	It <i>is</i> the <code>system</code> argument of every request — built at initialization, and the only text still guaranteed to be there on turn 200.

One model, one loop, many lists.

Every lever you have — system, tool, skill, sub-agent — is prose you wrote.

Bridge: Now Design and Dispatch Your Own

The lab makes every claim in this deck falsifiable in about forty minutes.

Part A — the mechanism, offline

- Build a messages list by hand and watch it grow
- Run T2a's loop against a scripted model — no key, no network, no cost
- Plot tokens per turn: see the cost curve
- Wire the three wirings from this deck and measure them

Part B — dispatch, live in your terminal

- Write three agent files: code-reviewer, domain-reviewer, verifier — T5b's RA team, made real
- Then write the fourth that team is missing: math-agent, from your filled canvas (T3a)
- Dispatch them at a flawed Aiyagari solver
- Run the four checks from the router section
- Narrow a description and watch dispatch stop
- See why *what an agent is allowed to read* decides what its review is worth
- Fill the 3×3 detection matrix and check the diagonal fires and the control row stays silent
- Then dispatch the same task at all three rungs, and script one

Bridge: Three Rungs, One Folder, and a Lab

Everything in this deck is one of two artifacts: a folder of Markdown files, or a loop over processes. Neither is a framework, and both are yours.

In the lab, added for this deck

- Dispatch the three background specialists — the prompt strings T3a asked for and never gave you
- Run one task at all three rungs and compare the transcripts
- Fan out read-only over every deck in the lecture and count unsourced numbers

Committed with this deck

- `deck-cartographer` — map a deck's structure and style
- `boundary-auditor` — say which deck owns a topic, with quotes
- `claim-checker` — find the source of every number, or flag it

The three briefs from the ledger that should have been files. At the repository root, so they are in scope where you actually work.

Then back to the spine. T4 comes next, then T5a returns to method — the homotopy workflow that decides *what* you should be asking any of these agents to do. A rung is a delivery mechanism; it does not improve the question.